

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0506

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and	BL4 – Analyze	5

	secondary indexing to support fast queries by department and CGPA range.		
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I G Vyshnavi (192525260) __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: vyshnavi

Q1. E-commerce Company – File Organization Strategy

An e-commerce company has **10 million product records**. The best choice is a **B+ Tree-based indexed file organization** with records stored in a suitable data file.

Why B+ Tree?

- Products need frequent **searching by ProductID, category, price, etc.**
- B+ Tree provides fast searching even when the database becomes very large.
- It supports both **exact-match and range queries**.
- Since index entries are ordered, queries such as *products between ₹500 and ₹1000* can be performed efficiently.
- The index can be stored separately from the actual product records.

Cost Analysis

Operation	Without Index	With B+ Tree
Search	$O(n)$	$O(\log n)$
Insert	Simple	$O(\log n)$
Range Search	Slow	Fast
Storage	Low	Extra index storage
Large Dataset	Poor performance	Good performance

For **10 million records**, scanning every record for each search would cause high disk I/O. A B+ Tree reduces the number of blocks that need to be accessed.

Conclusion:

Therefore, a **B+ Tree indexed organization** is suitable because it provides fast retrieval and efficient range searching. The additional index storage cost is justified by the significant reduction in search time.

Q2. Banking System – Indexing Strategy

The banking system needs fast retrieval using **Account Number** and **Date Range**.

Proposed Indexes

Use two indexes:

1. Primary/Unique Index on Account_Number

Account_Number $\rightarrow$ Transaction_Record

Account number is generally unique for an account, so a **B+ Tree index** can be created.

2. Composite B+ Tree Index

(Account_Number, Transaction_Date)

This supports queries such as:

SELECT *FROM TransactionsWHERE Account_Number = 101AND Transaction_Date BETWEEN '2026-01-01' AND '2026-01-31';

Why B+ Tree?

- Fast account-number lookup.
- Efficient date-range searching.
- Maintains keys in sorted order.
- Reduces disk I/O.
- Supports large transaction tables.

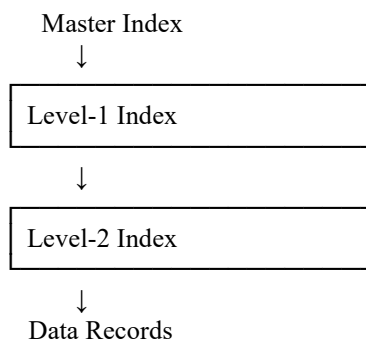
Conclusion

A B+ Tree index on Account_Number and a composite B+ Tree on (Account_Number, Transaction_Date) provide efficient transaction retrieval for both exact and range queries.

Q3. Healthcare System – Multi-Level Indexing

The healthcare system accesses patient records using **PatientID** and **Doctor Name**.

A **multi-level indexing structure** can be designed as follows:



Index Design

Primary index:

PatientID → Patient Record

Secondary index:

DoctorName → Patient Records

For example:

PatientID	Doctor
101	Dr. Ravi
102	Dr. Priya
103	Dr. Ravi

The secondary index can point to all patients associated with a particular doctor.

Advantages

- PatientID searches are fast.
- Doctor-based searches are also efficient.

- Multi-level indexing reduces disk accesses.
- Suitable for large healthcare databases.
- Indexes can be maintained separately from patient records.

Conclusion

A **multi-level B+ Tree-based indexing solution** with PatientID as the primary index and DoctorName as a secondary index provides efficient access through both search attributes.

Q4. Logistics – B-Tree vs B+ Tree

The logistics company performs **dynamic insertions and deletions** of shipment records.

Feature	B-Tree	B+ Tree
Data storage	Internal + leaf nodes	Leaf nodes
Search	Fast	Very fast
Range queries	Less efficient	Highly efficient
Sequential access	Moderate	Excellent
Dynamic insertion	Supported	Supported
Dynamic deletion	Supported	Supported

Analysis

Both B-Tree and B+ Tree automatically maintain balance after insertion and deletion.

However, **B+ Tree is more suitable** because:

1. All actual records are stored in leaf nodes.
2. Leaf nodes are linked sequentially.
3. Range searches can be performed efficiently.
4. Internal nodes contain only keys, allowing more keys per node.
5. More keys per node reduce the height of the tree.

Example

For finding shipments from:

Shipment 1000 to Shipment 2000

a B+ Tree can locate the first matching leaf and then follow the linked leaf nodes.

Conclusion

B+ Tree is the better choice for the logistics system because it handles dynamic updates efficiently and provides excellent sequential and range access.

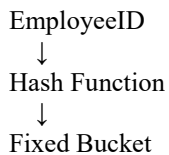
Q5. HR System – Static vs Extendible Hashing

The HR system contains **5000 employees**.

Hashing can directly map an EmployeeID to a storage bucket.

Static Hashing

A fixed number of buckets is created initially.



Advantages:

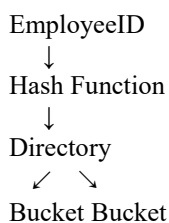
- Simple implementation.
- Fast exact-match search.
- Low management overhead.

Disadvantage:

- If employee records increase significantly, overflow buckets may be required.

Extendible Hashing

The number of buckets can dynamically increase.



Advantages:

- Handles database growth.
- Reduces overflow.
- Dynamically splits buckets.
- Good for frequently changing data.

Comparison:

Feature	Static Hashing	Extendible Hashing
Bucket size	Fixed	Dynamic
Growth handling	Poor	Excellent
Overflow	More likely	Reduced
Complexity	Low	Higher
HR system suitability	Moderate	High

Conclusion

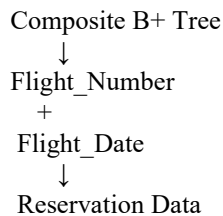
For an HR system where employee records may grow, **extendible hashing is preferable** because it dynamically adapts to changes in the number of records.

Q6. Airline Reservation – Combined Index

The airline system requires:

- **Point queries by Flight Number**
- **Range queries by Date**

A combined indexing strategy can use a **composite B+ Tree index**.



Example

Suppose:

Flight No. = AI101
Date = 20-Aug-2026

The index can quickly locate the required flight.

For a date-range query:

Flight_Date BETWEEN
01-Aug-2026 AND 31-Aug-2026

the ordered B+ Tree allows efficient traversal of matching entries.

Benefits

- Fast exact flight-number search.
- Efficient date-range search.
- Reduced disk access.
- Suitable for large reservation databases.
- Supports both query types using indexed ordering.

Conclusion

A composite B+ Tree index involving Flight_Number and Flight_Date is an effective solution for supporting both point and range queries.

Q7. University Database – File Organization and Secondary Index

A university database needs fast queries by **Department** and **CGPA range**.

File Organization

A **sorted file organization** can be used with StudentID or another suitable primary field.

Student Records



Sorted File



Secondary Indexes

Secondary Index 1 – Department

Department → Student Records

Example:

CSE → S101, S105, S110

ECE → S102, S107

Secondary Index 2 – CGPA

CGPA → Student Records

Since CGPA queries may involve ranges such as:

CGPA BETWEEN 8.0 AND 9.0

a **B+ Tree secondary index** is suitable.

Advantages

- Department-based searches become faster.
- CGPA range queries are efficient.
- B+ Tree maintains sorted index entries.
- Reduces complete table scans.
- Suitable for large student databases.

Conclusion

A sorted file organization with secondary B+ Tree indexes on Department and CGPA provides efficient access to student records.

Q8. Disk Block Utilization

Given:

- Number of records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4096 bytes**

Step 1: Records per Block

Records per block = $4096 / 50 = 81.92$

Therefore, only **81 complete records** can fit in one block.

Step 2: Required Blocks

Blocks = $100000 / 81 = 1235$ blocks

Step 3: Actual Utilization

Each block stores:

$81 \times 50 = 4050$ bytes

Unused space:

$4096 - 4050 = 46$ bytes

Utilization:

$4096 / 4050 \times 100 \approx 98.88\%$

Comparison

File Organization	Storage	Search	Insert
Heap	High utilization	Slow	Fast
Sorted	High utilization	Fast	Slower
Hashed	Good	Very fast exact search	Fast

Conclusion

With 50-byte records and 4KB blocks, approximately **81 records fit per block**, giving about **98.88% block utilization**. The choice between Heap, Sorted and Hashed depends mainly on the type of query required.

Q9. Social Media – Composite Indexing Strategy

The platform contains **500 million posts** and needs indexing by:

- user_id
- timestamp
- hashtag

A **composite indexing strategy** can be used.

Proposed Indexes

Index 1:

(user_id, timestamp)

Index 2:

(hashtag, timestamp)

Index 1 – User + Timestamp

Useful for queries such as:

```
SELECT * FROM Posts WHERE user_id = 101 ORDER BY timestamp;
```

It quickly finds posts belonging to a particular user and orders them by time.

Index 2 – Hashtag + Timestamp

Useful for:

```
SELECT *FROM Posts WHERE hashtag = '#technology'ORDER BY timestamp;
```

It quickly retrieves posts containing a particular hashtag.

Why Composite Indexing?

- Supports multiple query conditions.
- Reduces full-table scans.
- Improves timestamp-based retrieval.
- Suitable for massive datasets.
- B+ Tree indexes can efficiently support ordered timestamp searches.

Conclusion

For 500 million posts, composite B+ Tree indexes on (user_id, timestamp) and (hashtag, timestamp) provide efficient retrieval while avoiding unnecessary scanning of the entire dataset.

Q10. Supermarket Billing System – Performance Comparison

The supermarket processes **1 million transactions daily**.

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	Fast	Slow	Fast
Delete	Moderate	Slow	Fast
Exact Search	Slow	Fast	Very Fast
Range Search	Slow	Very Fast	Poor
Maintenance	Easy	High	Moderate

Heap File

Records are stored wherever space is available.

Advantage: Very fast insertion.

Disadvantage: Searching may require scanning many records.

Sorted File

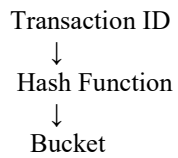
Records are maintained in sorted order.

Advantage: Excellent for searching and range queries.

Disadvantage: Inserting new transactions may require shifting records or reorganizing the file.

Hashed File

A hash function determines the location of a transaction.



Advantage: Very fast exact-match searches and insertions.

Disadvantage: Not suitable for range queries.

Final Recommendation

For a supermarket billing system where transactions are frequently inserted and exact transaction/bill searches are common, Hashed File Organization is the best choice.

If frequent range queries such as daily/monthly transaction analysis are required, a Sorted File or B+ Tree index can be added.